What is an Interrupt ?

Interrupt is a signal/event given to any CPU to break its normal flow of execution and redirect to another flow of execution with higher priority than the current flow of execution.And after completing the flow of execution with higher priority return back and continue with its normal flow of execution.

Types of Interrupts :

Hardware Interrupts : Refers to Interrupts which can be Generated by External Devices/Peripheral / OnChip Peripherals onto the CPU. And these these types of interrupts can also be referred to as Asynchronous Interrupts.

Software Interrupts : Refers to Interrupts which can be Generated by CPU when it is executing a particular instruction during flow of execution. These types of interrupts occurs in an orderly/predictible way. And hence can also be referred to as Synchronous Events.

And these are in someway similar to Calling of Sub-Routines But Normal Sub-Routines are handled And How Interrupt Sub-Routines are Handled differ slightly.

Maskable Interrupts – are those set of interrupts within/supported by CPU which can be blocked/ignored and also unblocked/considered for being serviced or not. Here the programer has the privilidge to configure the interrupts to be blocked/unblocked. (USER/Peripheral Interrupts)

Non Maskable Interrupts – are those interrupts which cannot be blocked by the programmer & will definitely be serviced when such interrupts are raised. (RESET for any CPU)

Interrupt Service Routine/Interrupt Sub Routine/Interrupt Handler (ISR) – refer to a block of statements/code defined somewhere in code/rom section which executes in response to a particular interrupt raised onto CPU.

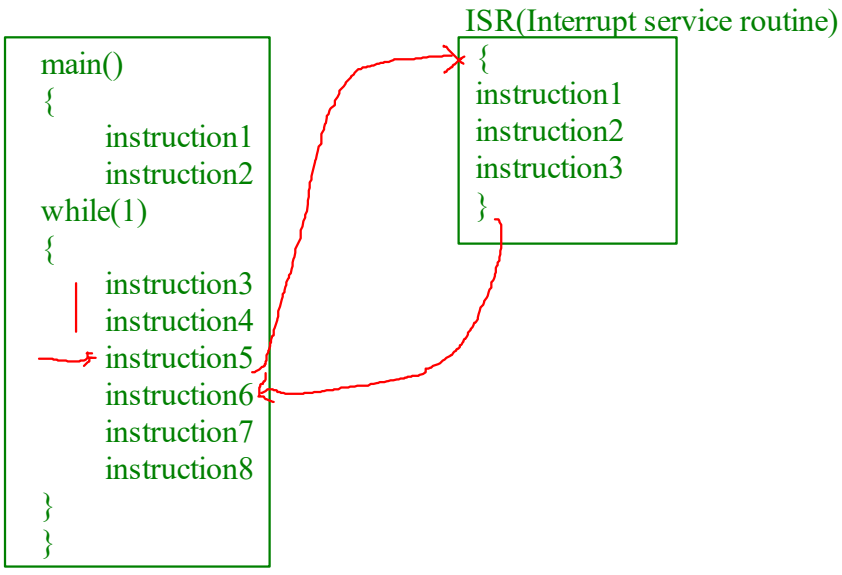
Interrupt Latency (Time) – Refers to the time lapse/gap between when the interrupt was raised onto the CPU and when did the CPU actually see/sense/check/poll that interrupt signal is available.

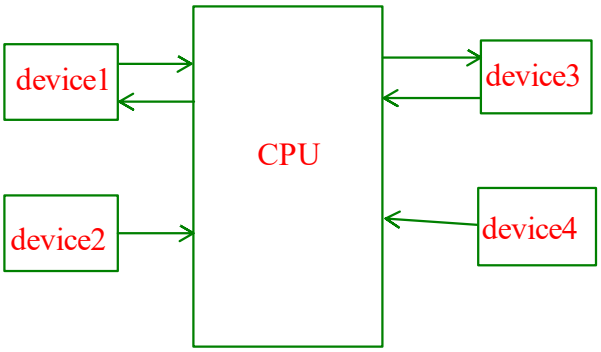
Interrupt Response Time – Refers to the ILT + Time for Context Saving (Context Switch) (Default Context Saving + User Define Context Saving)+Time to Executing the 1st Instruction related to the activity of ISR.

Context Saving (Context Switching) –refers to saving/pushing of return address,certain registers & flags information onto stack, because those registers & flags may be also required in the ISR, and after completing the ISR the return address helps returning back to the suspended normal flow of execution with the registers & flag restored to continue with. Context Saving Operation consumes time which should be also considered.

Interrupt Recovery Time – Refer to Time of Context Retrieving (Context Switch) & continuing from the point where normal flow of execution was suspended from.

Context Retrieving (Context Switching) – refers to poping/retrieving the certain registers & flags information & return address to get back/resume execution from the point where it was suspended in the normal flow of execution.





In interrupt method
which ever the device
sends a request signal
that device gets the service.

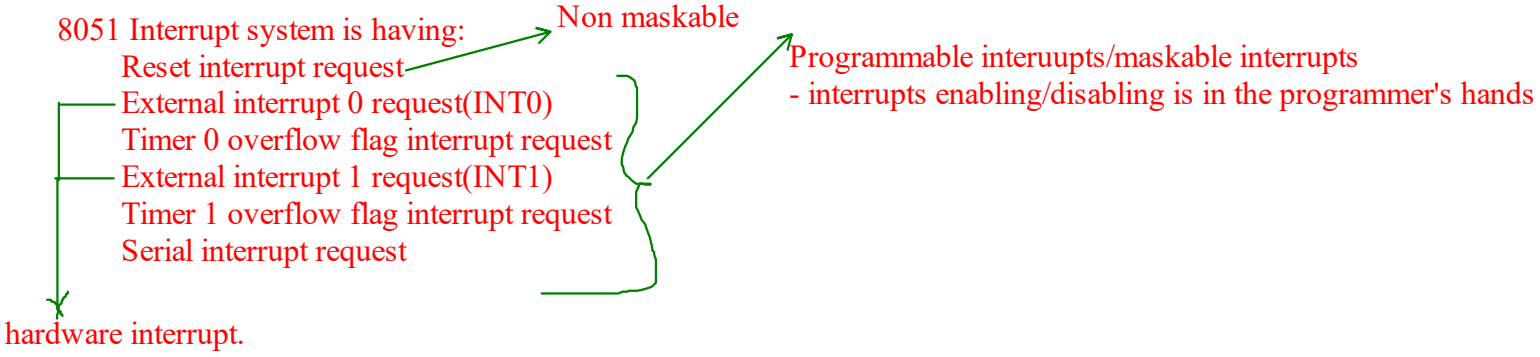
Polling:

```
Main()
{
      some instructions
      while(1)
      {
            ------
            ------
            ------
            ------
            if(sw is pressed)
            {
                  -------
                  -------
            }
            --------
            --------
      }
}
```

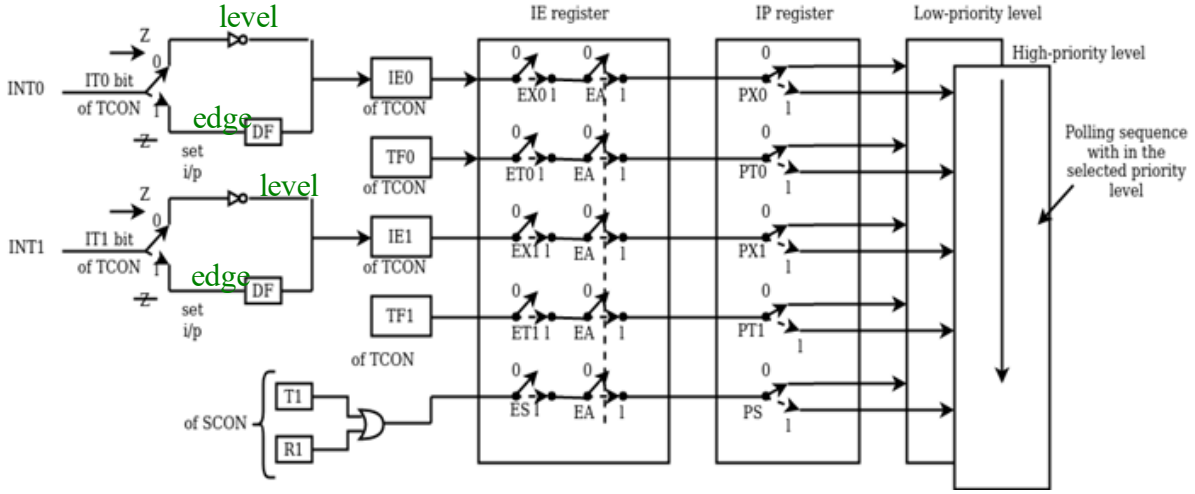
Interrupt:

```
Void interrupt service routine
{
      ----
      ----
}
Main()
{
      some instructions
      while(1)
      {
            ------
            ------
            ------
            ------
            ------
            -----
      }
}
```

What happens when a vectored interrupt signal is given to the CPU?
Sequential steps:
1. After interrupt signal is given to the CPU, CPU will recognise that interrupt signal.
2. Completes the current instruction's execution.
3. Saves the current task status (Next instruction addr,interrupt flag status,lower priority interrupts gets disabled.)
4. PC (program counter) is loaded with Vector_addr of the respective interrupt signal.
5. ISR is called from vector addr and executed.
6. Returns control to the interrupted code by retrieving return addr,interrupt flags and enable the interrupts whichever are disabled .
7. Then continues to execute regular task.



level - only low level
edge - falling edge (high to low)



IE Register: Interrupt Enable Register

EA	---	ET2	ES	ET1	EX1	ET0	EX0

EA : Global interrupt enable bit
ET2 : Timer2 Enable bit
ES : Serial interrupt enable bit
ET1 : Timer 1 interrupt enable bit
EX1 : External interrupt 1 enable bit
ET0 : Timer 0 interrupt enable bit
EX0 : External interrupt 0 enable bit

0 = Disabled 1 = Enabled

IP: Interrupt Priority Register

---	---	---	PT2	PS	PT1	PX1	PT0	PX0

PT2 : Timer 2
PS : Serial interface
PT1 : Timer 1
PX1 : External interrupt 1
PT0 : Timer 0
PX0 : External interrupt 0

0 = Low priority 1 = High priority

Source	Address
RESET	0000H
INT0	0003H
T0	000BH
INT1	0013H
T1	001BH
RI&TI	0023H

interrupt0 vector address is starting from 0003H. why?
due to LJMP instruction - 3 bytes instruction

one interrupt vector address to other interrupt vector address gap is 8 bytes.
if interrupt service routine logic is more that 8 bytes what will will happen?
Ans: compiler will take care

RET - returning from normal subroutine/function
RETI - returning from interrupt service routine

If higher priority interrupt req comes when CPU is executing lower priority interrupt service routine then control need to be transferred to the high priority interrupt req which is called as nested interrpt mechanism.

NOTE: To change the priority order or to work nested interrupt mechanism IP register need to be accessed.
IP register (bits) must be either 0 or only with one bit set.

IP=0X00	PX0=1	PT0=1	PX1=1	PT1=1	PS=1
IE0	IE0	TF0	IE1	TF1	RI&TI
TF0	TF0	IE0	IE0	IE0	IE0
IE1	IE1	IE1	TF0	TF0	TF0
TF1	TF1	TF1	TF1	IE1	IE1
RI&TI	RI&TI	RI&TI	RI&TI	RI&TI	TF1

dec order of priority

Interrupt service routine syntax:

```
void isr_name(void) interrupt IID(Interrupt identifier)
{
      ------
}
```

It tells to the compiler that the service routine is for which interrupt request.

IID -
0 - int0
1 - timer0
2 - int1
3 - timer1
4 - serial

Interrupt keyword tells to the compiler that it is a Interrupt service routine.

During An On Ongoing Instruction (of 1MC/2MC/4MC) is executing and any interrupt is raised before/on S5P2(state5 pulse2) of any MC,then interrupt is seen by CPU.
CPU Completes the Ongoing Instruction.
Save Return Address of Next Instruction available/pointed by PC saved onto Stack (using Push Operations PUSH PCL, PUSH PCH) - this is called “Default Context Saving”
Perform an LCALL to Interrupt Vector thereby branching to ISR
Executes statements with ISR until Last statement (RETI)
On RETI Instruction Being Executed, CPU Retreives Return Address from Stack(using POP PCH,POP PCL) into PC – this is called “Default Context Retreiving” and hence making CPU to Continue from where it was suspended in main-line execution flow.

RESET:
- Non maskable interrupt
- Atleast high for 2 machine cycles
- Reset address - 0000h
- Bank 0 is selected
- Internal RAM contents may change